

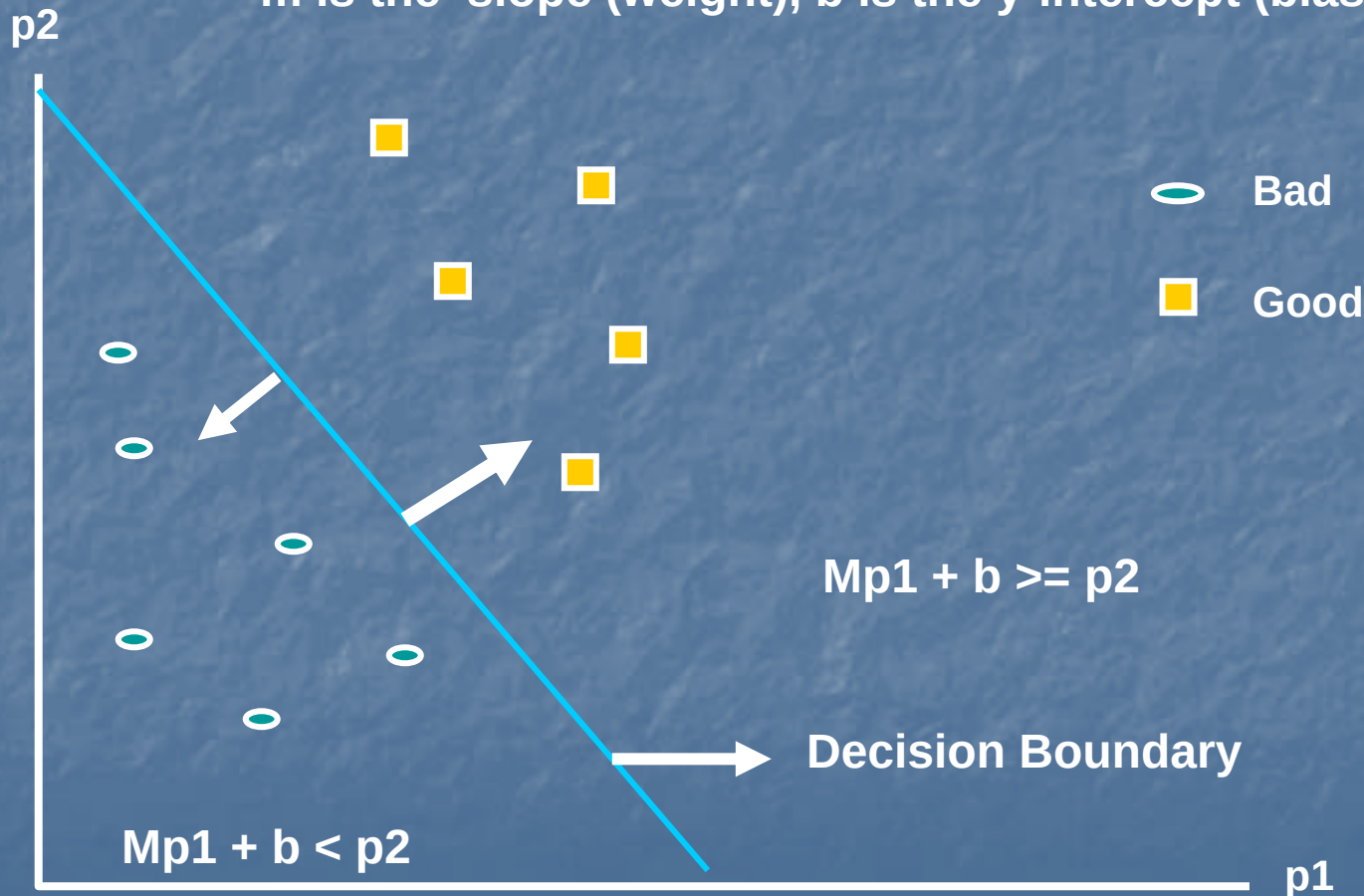
An Illustrative Example

Simple Neural Network

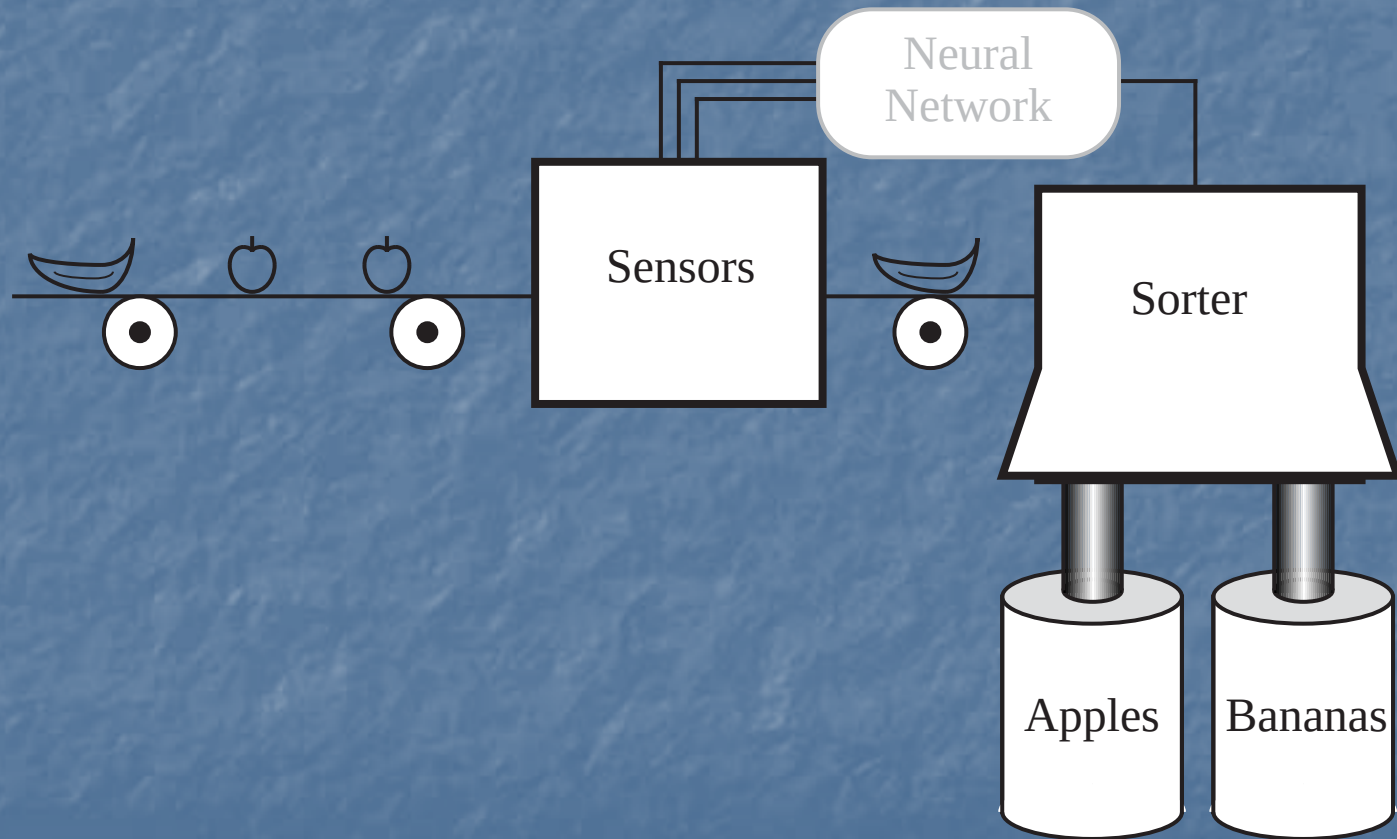
One neuron with a linear activation function => Straight Line

Recall the equation of a straight Line: $y = mx + b$

m is the slope (weight), b is the y-intercept (bias)



Apple / Banana Sorter



Prototype Vectors

Measurement
Vector

$$\mathbf{p} = \begin{bmatrix} \text{shape} \\ \text{texture} \\ \text{weight} \end{bmatrix}$$

Prototype Banana

$$\mathbf{p}_1 = \begin{bmatrix} -1 \\ 1 \\ -1 \end{bmatrix}$$

Prototype Apple

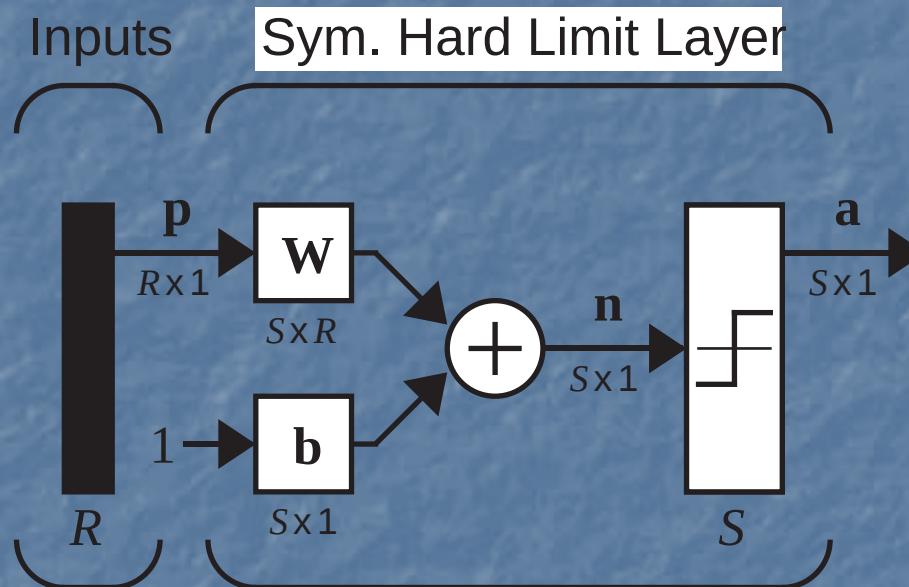
$$\mathbf{p}_2 = \begin{bmatrix} 1 \\ 1 \\ -1 \end{bmatrix}$$

Shape: {1: round ; -1: elliptical}

Texture: {1: smooth ; -1: rough}

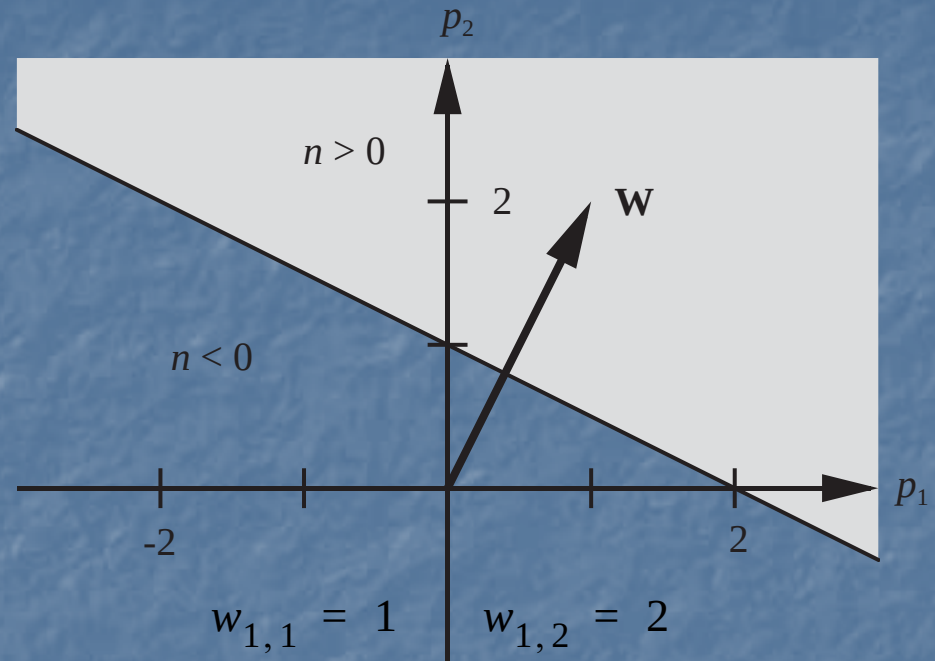
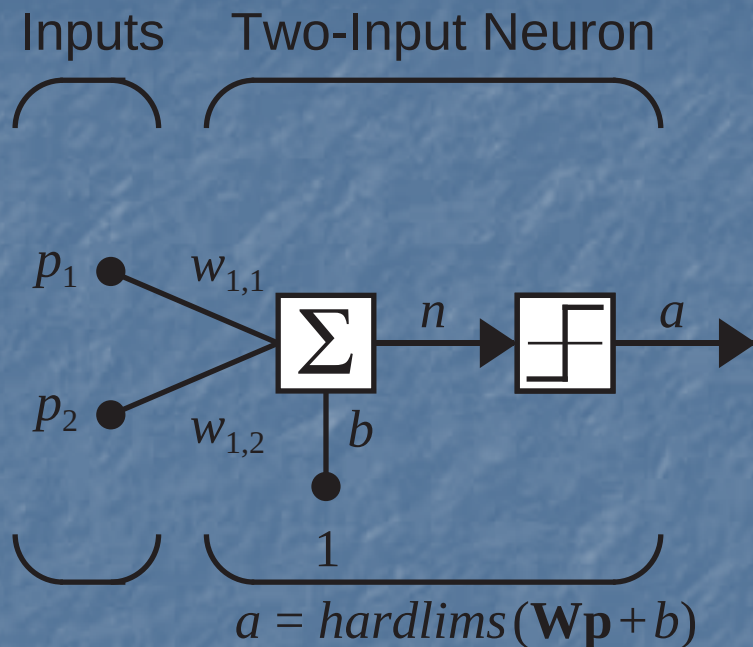
Weight: {1: > 1 lb. ; -1: < 1 lb.}

Perceptron



$$\mathbf{a} = \text{hardlims}(\mathbf{W}\mathbf{p} + \mathbf{b})$$

Two-Input Case



$$a = \text{hardlims}(n) = \text{hardlims}\left(\begin{bmatrix} 1 & 2 \end{bmatrix} \mathbf{p} + (-2)\right)$$

Decision Boundary

$$\mathbf{W}\mathbf{p} + b = 0 \quad \begin{bmatrix} 1 & 2 \end{bmatrix} \mathbf{p} + (-2) = 0$$

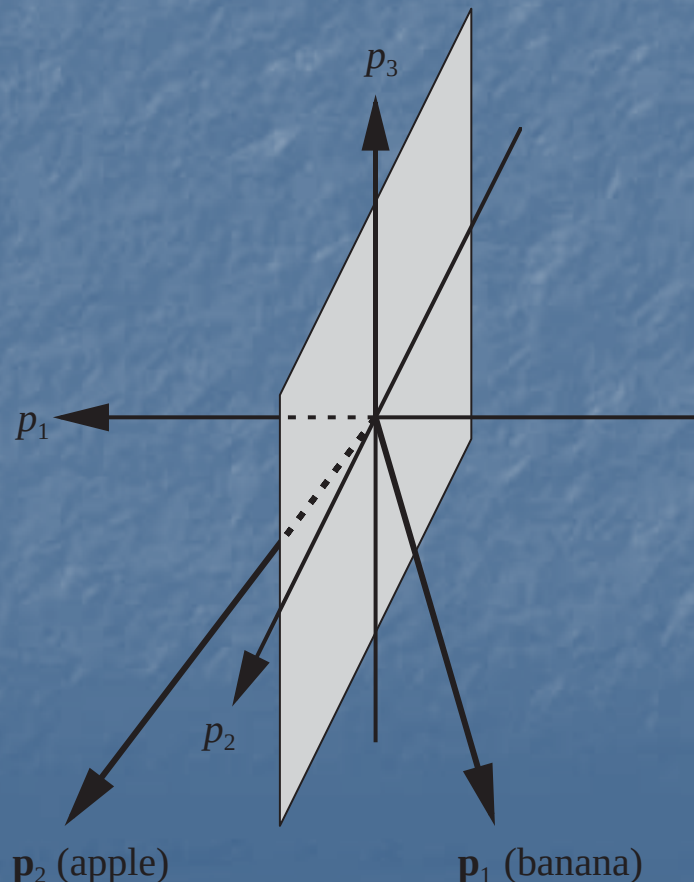
Apple / Banana Example

$$a = \text{hardlims} \left(\begin{bmatrix} w_{1,1} & w_{1,2} & w_{1,3} \end{bmatrix} \begin{bmatrix} p_1 \\ p_2 \\ p_3 \end{bmatrix} + b \right)$$

The decision boundary should separate the prototype vectors.

$$p_1 = 0$$

The weight vector should be orthogonal to the decision boundary, and should point in the direction of the vector which should produce an output of 1. The bias determines the position of the boundary



$$\begin{bmatrix} -1 & 0 & 0 \end{bmatrix} \begin{bmatrix} p_1 \\ p_2 \\ p_3 \end{bmatrix} + 0 = 0$$

Testing the Network

Banana:

$$a = \text{hardlims} \left(\begin{bmatrix} -1 & 0 & 0 \end{bmatrix} \begin{bmatrix} -1 \\ 1 \\ -1 \end{bmatrix} + 0 \right) = 1(\text{banana})$$

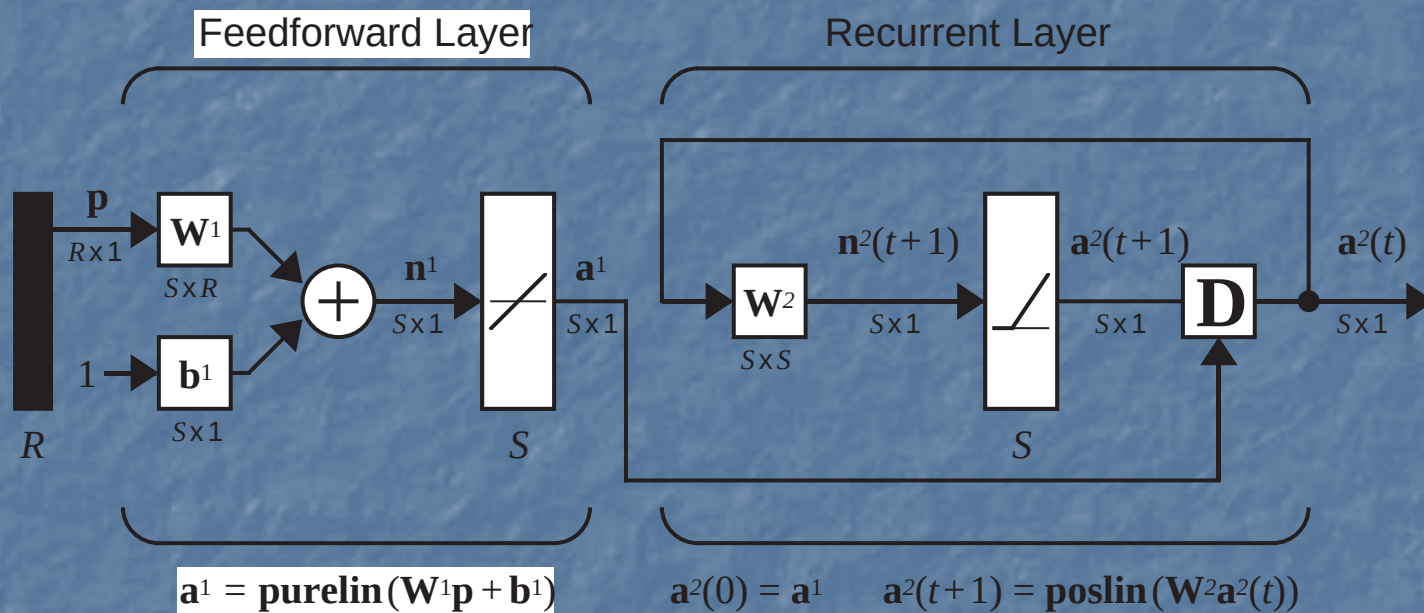
Apple:

$$a = \text{hardlims} \left(\begin{bmatrix} -1 & 0 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ -1 \end{bmatrix} + 0 \right) = -1(\text{apple})$$

“Rough” Banana:

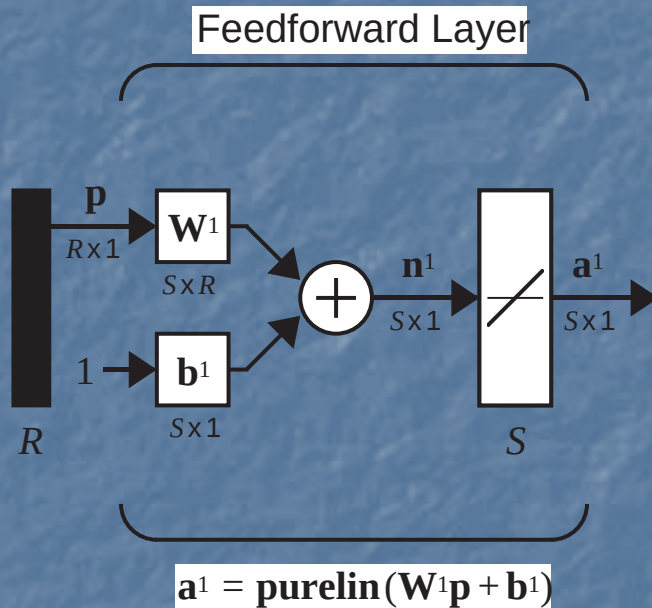
$$a = \text{hardlims} \left(\begin{bmatrix} -1 & 0 & 0 \end{bmatrix} \begin{bmatrix} -1 \\ -1 \\ -1 \end{bmatrix} + 0 \right) = 1(\text{banana})$$

Hamming Network



Feedforward Layer

For Banana/Apple Recognition



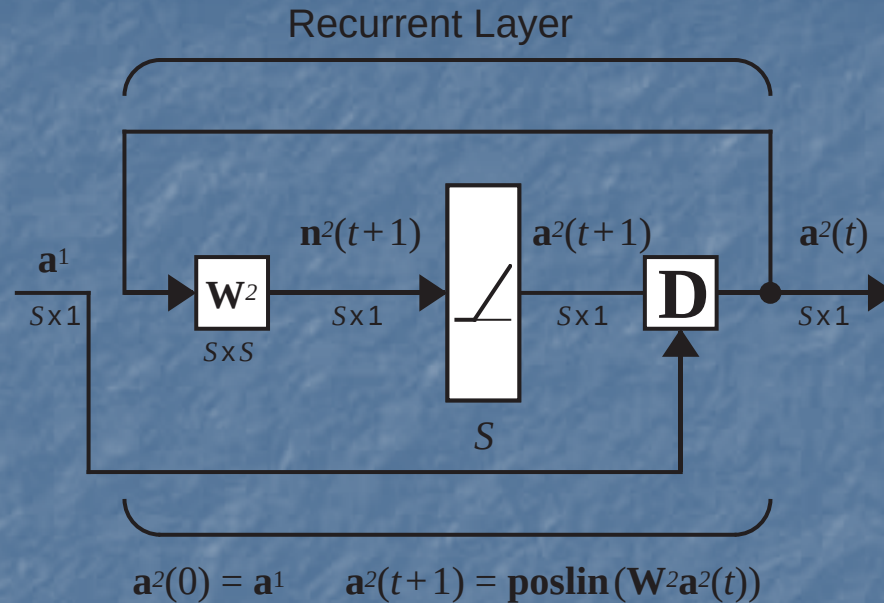
$$S = 2$$

$$\mathbf{W}^1 = \begin{bmatrix} \mathbf{p}_1^T \\ \mathbf{p}_2^T \end{bmatrix} = \begin{bmatrix} -1 & 1 & -1 \\ 1 & 1 & -1 \end{bmatrix}$$

$$\mathbf{b}^1 = \begin{bmatrix} R \\ R \end{bmatrix} = \begin{bmatrix} 3 \\ 3 \end{bmatrix}$$

$$\mathbf{a}^1 = \mathbf{W}^1 \mathbf{p} + \mathbf{b}^1 = \begin{bmatrix} \mathbf{p}_1^T \\ \mathbf{p}_2^T \end{bmatrix} \mathbf{p} + \begin{bmatrix} 3 \\ 3 \end{bmatrix} = \begin{bmatrix} \mathbf{p}_1^T \mathbf{p} + 3 \\ \mathbf{p}_2^T \mathbf{p} + 3 \end{bmatrix}$$

Recurrent Layer



$$\mathbf{W}^2 = \begin{bmatrix} 1 & -\varepsilon \\ -\varepsilon & 1 \end{bmatrix} \quad \varepsilon < \frac{1}{S-1}$$

$$\mathbf{a}^2(t+1) = \text{poslin} \left(\begin{bmatrix} 1 & -\varepsilon \\ -\varepsilon & 1 \end{bmatrix} \mathbf{a}^2(t) \right) = \text{poslin} \left(\begin{bmatrix} a_1^2(t) - \varepsilon a_2^2(t) \\ a_2^2(t) - \varepsilon a_1^2(t) \end{bmatrix} \right)$$

Hamming Operation

First Layer

Input (Rough Banana)

$$\mathbf{p} = \begin{bmatrix} -1 \\ -1 \\ -1 \end{bmatrix}$$

$$\mathbf{a}^1 = \begin{bmatrix} -1 & 1 & -1 \\ 1 & 1 & -1 \end{bmatrix} \begin{bmatrix} -1 \\ -1 \\ -1 \end{bmatrix} + \begin{bmatrix} 3 \\ 3 \end{bmatrix} = \begin{bmatrix} (1 + 3) \\ (-1 + 3) \end{bmatrix} = \begin{bmatrix} 4 \\ 2 \end{bmatrix}$$

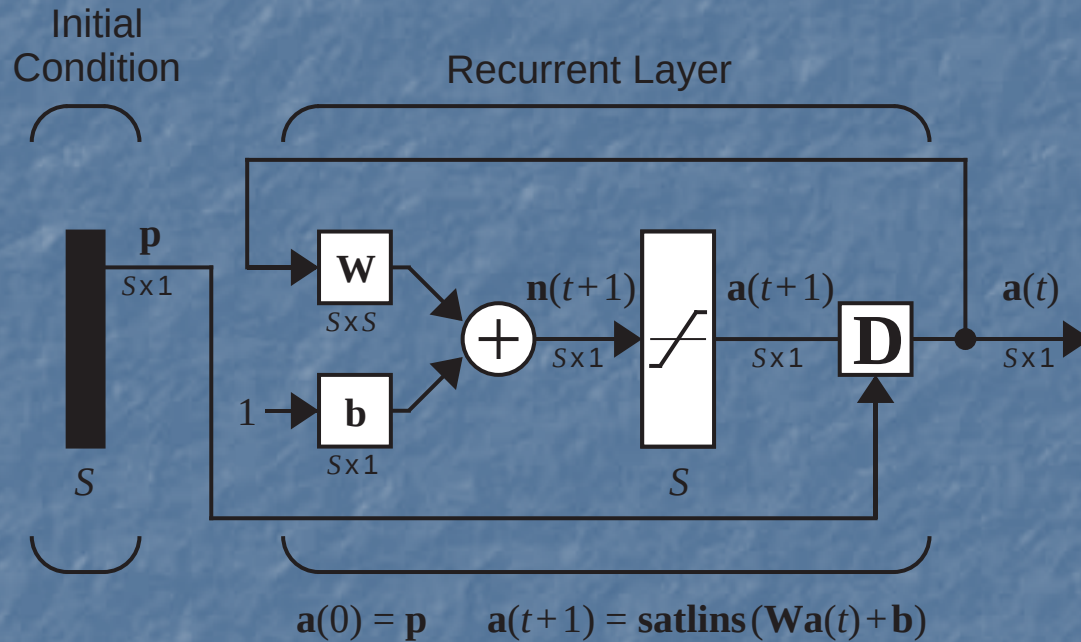
Hamming Operation

Second Layer

$$\mathbf{a}^2(1) = \mathbf{poslin}(\mathbf{W}^2 \mathbf{a}^2(0)) = \begin{cases} \mathbf{poslin}\left(\begin{bmatrix} 1 & -0.5 \\ -0.5 & 1 \end{bmatrix} \begin{bmatrix} 4 \\ 2 \end{bmatrix}\right) \\ \mathbf{poslin}\left(\begin{bmatrix} 3 \\ 0 \end{bmatrix}\right) = \begin{bmatrix} 3 \\ 0 \end{bmatrix} \end{cases}$$

$$\mathbf{a}^2(2) = \mathbf{poslin}(\mathbf{W}^2 \mathbf{a}^2(1)) = \begin{cases} \mathbf{poslin}\left(\begin{bmatrix} 1 & -0.5 \\ -0.5 & 1 \end{bmatrix} \begin{bmatrix} 3 \\ 0 \end{bmatrix}\right) \\ \mathbf{poslin}\left(\begin{bmatrix} 3 \\ -1.5 \end{bmatrix}\right) = \begin{bmatrix} 3 \\ 0 \end{bmatrix} \end{cases}$$

Hopfield Network



Apple / Banana Problem

$$\mathbf{W} = \begin{bmatrix} 1.2 & 0 & 0 \\ 0 & 0.2 & 0 \\ 0 & 0 & 0.2 \end{bmatrix}, \mathbf{b} = \begin{bmatrix} 0 \\ 0.9 \\ -0.9 \end{bmatrix}$$

$$a_1(t+1) = \text{satlins}(1.2a_1(t))$$

$$a_2(t+1) = \text{satlins}(0.2a_2(t) + 0.9)$$

$$a_3(t+1) = \text{satlins}(0.2a_3(t) - 0.9)$$

Test: “Rough” Banana

$$\mathbf{a}(0) = \begin{bmatrix} -1 \\ -1 \\ -1 \end{bmatrix}$$

$$\mathbf{a}(1) = \begin{bmatrix} -1 \\ 0.7 \\ -1 \end{bmatrix}$$

$$\mathbf{a}(2) = \begin{bmatrix} -1 \\ 1 \\ -1 \end{bmatrix}$$

$$\mathbf{a}(3) = \begin{bmatrix} -1 \\ 1 \\ -1 \end{bmatrix} \text{ (Banana)}$$

Summary

- **Perceptron**
 - **Feedforward Network**
 - **Linear Decision Boundary**
 - **One Neuron for Each Decision**
- **Hamming Network**
 - **Competitive Network**
 - **First Layer – Pattern Matching (Inner Product)**
 - **Second Layer – Competition (Winner-Take-All)**
 - **# Neurons = # Prototype Patterns**
- **Hopfield Network**
 - **Dynamic Associative Memory Network**
 - **Network Output Converges to a Prototype Pattern**
 - **# Neurons = # Elements in each Prototype Pattern**